

RipenTom: Sistema Inteligente de Clasificación de Tomates por Estado de Madurez Mediante Visión por Computadora

Juliana Sánchez Gómez*, Juan Diego Collazos Mejía†, Ángel Luis Obando Fajardo‡
Juan José Torres Murillo§, Marco Riascos Salguero¶

Programa de Ingeniería de Sistemas y Computación
Pontificia Universidad Javeriana Cali
Cali, Valle del Cauca, Colombia

Resumen—Este artículo presenta RipenTom, un sistema inteligente para la clasificación automática del estado de madurez de tomates en tres categorías: *ripe* (maduro), *unripe* (inmaduro) y *damaged* (dañado). El sistema integra técnicas de aprendizaje automático clásico y aprendizaje profundo, con un enfoque novedoso de extracción de características basado en múltiples espacios de color, texturas y análisis por cuadrantes. El conjunto de datos fue curado manualmente eliminando muestras incorrectamente etiquetadas y balanceado a 400 imágenes por clase mediante aumentación controlada. Se evaluaron cuatro algoritmos de ML clásico (SVM, Random Forest, Gradient Boosting y KNN) y una red neuronal convolucional profunda (MobileNetV3-Small con fine-tuning). En el enfoque de extracción directa de características (100 features, vector único por imagen), Random Forest obtuvo el mejor desempeño con *accuracy* de 0.853 y F1-macro de 0.853. Con la estrategia de análisis por cuadrantes 2x2 (200 features), SVM-RBF alcanzó 0.867 de *accuracy* y 0.865 de F1-macro, siendo el modelo con mayor rendimiento general. El sistema fue desplegado como aplicación web con frontend estático en Vercel y backend Flask-API, permitiendo la clasificación en tiempo real de imágenes subidas por el usuario. Los resultados demuestran que el aprendizaje automático clásico con ingeniería de características robusta supera al modelo de aprendizaje profundo en este dominio específico con datos limitados.

Index Terms—clasificación de tomates, visión por computadora, Random Forest, MobileNetV3, ingeniería de características, análisis por cuadrantes, aprendizaje automático, sistemas inteligentes

I. INTRODUCCIÓN

La clasificación automática del estado de madurez de frutas y hortalizas representa uno de los problemas más relevantes en la intersección entre la agricultura de precisión y la inteligencia artificial. En el contexto de la cadena agroindustrial, la identificación manual del punto óptimo de cosecha y la detección de producto dañado conllevan costos operativos significativos, alta variabilidad subjetiva entre operarios y riesgos sanitarios asociados al procesamiento de producto en mal estado [1]. El tomate resulta especialmente relevante dado su amplio consumo global y la sensibilidad de su calidad a condiciones de manejo postcosecha [2].

La visión por computadora ha emergido como una solución escalable y reproducible para automatizar estas tareas

de inspección, extrayendo información objetiva de imágenes digitales sin las limitaciones de fatiga o sesgo propias del juicio humano [3]. Sin embargo, la aplicación práctica de estos sistemas enfrenta desafíos no triviales: la construcción de conjuntos de datos de calidad suficiente, el diseño de características discriminativas que capturen las propiedades visuales relevantes, y la selección del modelo de aprendizaje que mejor generalice al problema concreto.

Este trabajo presenta **RipenTom**, un sistema completo de clasificación de tomates en tres estados (maduro (*ripe*), inmaduro (*unripe*) y dañado (*damaged*)) que aborda estos desafíos desde un enfoque iterativo y basado en evidencia experimental. El desarrollo del proyecto atravesó varias etapas críticas de decisión: la curación manual de un conjunto de datos inicialmente ruidoso, el diseño y expansión progresiva del vector de características, la incorporación de una estrategia de análisis por cuadrantes espaciales, y la comparación sistemática entre algoritmos de aprendizaje automático clásico y redes neuronales convolucionales profundas.

Las contribuciones principales de este trabajo son: (1) un protocolo de curación y balanceo de datos específico para clasificación de tomates; (2) un vector de características de 100 dimensiones que combina estadísticas de color en espacios HSV y LAB, histogramas de cromaticidad, texturas GLCM y LBP, y descriptores de defectos superficiales; (3) una estrategia de análisis por cuadrantes 2x2 que amplía el vector a 200 dimensiones incorporando información espacial local; y (4) un sistema de despliegue web funcional que permite clasificar imágenes en tiempo real.

El resto del artículo se organiza de la siguiente manera: la Sección II revisa el estado del arte en clasificación de frutas por visión por computadora. La Sección III describe la metodología experimental, incluyendo la construcción del dataset, la extracción de características y los modelos evaluados. La Sección IV detalla el diseño del sistema y su arquitectura de despliegue. La Sección V presenta los resultados experimentales. La Sección VI incluye la discusión ética. Finalmente, la Sección VII presenta las conclusiones y trabajo futuro.

II. ESTADO DEL ARTE

II-A. Clasificación de frutas mediante aprendizaje automático clásico

Los enfoques basados en extracción manual de características han dominado durante décadas la clasificación de frutas por visión por computadora. Kondo et al. [4] demostraron que estadísticas de color en espacios cromáticos como HSV y CIELab permiten discriminar de manera efectiva entre estados de madurez en tomates, dado que el proceso de maduración implica transformaciones bioquímicas (principalmente la degradación de clorofila y síntesis de licopeno) que se manifiestan directamente en el matiz y la saturación de la imagen. Payne et al. [5] establecieron que la combinación de características de color con descriptores de textura mejora significativamente la discriminación en comparación con el uso exclusivo de información cromática.

Los descriptores de textura han sido particularmente útiles para identificar daños superficiales. Las matrices de coocurrencia de niveles de gris (GLCM, *Gray-Level Co-occurrence Matrix*) permiten cuantificar propiedades como contraste, correlación, energía y homogeneidad de la textura superficial [6]. Los patrones binarios locales (LBP, *Local Binary Pattern*) constituyen una alternativa computacionalmente eficiente y robusta a cambios de iluminación [7]. En el contexto de detección de defectos en frutas, la combinación de ambos descriptores ha reportado resultados consistentemente superiores al uso aislado de características de color [8].

II-B. Redes neuronales convolucionales en inspección de calidad

El surgimiento de las redes neuronales convolucionales profundas (CNN) y su aplicación al reconocimiento de imágenes ha transformado el campo de la inspección visual automatizada. Arquitecturas como VGG16, ResNet y MobileNet, preentrenadas sobre ImageNet, han sido adaptadas mediante transferencia de aprendizaje (*transfer learning*) para tareas de clasificación de frutas con conjuntos de datos relativamente pequeños [9].

Mohanty et al. [10] demostraron que CNN preentrenadas pueden alcanzar precisiones superiores al 99 % en clasificación de enfermedades foliares, aunque estos resultados corresponden a condiciones controladas de laboratorio. En escenarios más realistas, con variabilidad de iluminación, fondos complejos y datasets pequeños, el rendimiento de las CNN tiende a degradarse significativamente [11]. MobileNetV3-Small, diseñada específicamente para entornos de recursos limitados [12], ha mostrado ser una alternativa equilibrada entre eficiencia computacional y capacidad representacional.

II-C. Importancia de la calidad del dataset

Un factor determinante en el rendimiento de cualquier sistema de clasificación es la calidad del conjunto de entrenamiento. Ding [13] documenta que errores sistemáticos de etiquetado pueden degradar la accuracy en hasta un 15 % incluso con arquitecturas sofisticadas. El desbalanceo de clases constituye otro factor crítico: modelos entrenados sobre

datasets desbalanceados tienden a sesgar sus predicciones hacia la clase mayoritaria [14]. Técnicas de aumentación de datos (rotación, volteo horizontal/vertical, ajuste de brillo y contraste) son comúnmente empleadas para expandir conjuntos de datos pequeños manteniendo la distribución de clases [15].

II-D. Análisis espacial local en imágenes de frutas

La división de imágenes en regiones o cuadrantes para el análisis local ha sido explorada en problemas de detección de defectos superficiales. Riquelme et al. [16] proponen que el análisis de zonas locales permite detectar irregularidades que el procesamiento global de la imagen tiende a enmascarar al promediar regiones sanas y dañadas. Este enfoque resulta especialmente relevante para la clase *damaged*, donde los defectos suelen ser localizados y no distribuidos uniformemente sobre la superficie del fruto.

III. METODOLOGÍA

III-A. Construcción y curación del dataset

El dataset base utilizado en este trabajo fue obtenido de una colección pública disponible en Kaggle [17], compuesta por imágenes etiquetadas en las categorías *ripe*, *unripe* y *damaged*. Sin embargo, una inspección inicial reveló problemas críticos de calidad: imágenes de tomates maduros incorrectamente clasificadas como dañados, imágenes duplicadas o casi-duplicadas que inflarían artificialmente las métricas de validación, y ejemplos ambiguos que no correspondían claramente a ninguna clase.

Para resolver estos problemas se adoptó un protocolo de curación manual sistemático: cada imagen del dataset fue revisada individualmente, y se tomó una decisión explícita de reasignación de clase o eliminación cuando la etiqueta original era incorrecta. Tras la curación, las tres clases presentaban tamaños desiguales. Para restaurar el balance sin perder información, se aplicó aumentación controlada sobre la clase deficitaria mediante transformaciones de bajo sesgo semántico: rotaciones aleatoria de hasta 15°, volteo horizontal y vertical, y ajustes de brillo/contraste. El dataset final balanceado contiene **400 imágenes por clase** (1.200 imágenes totales), divididas en proporción 70/30 para entrenamiento y validación, respetando la estratificación por clase.

Cuadro I
DISTRIBUCIÓN DEL DATASET TRAS CURACIÓN Y BALANCEO

Clase	Total	Train (70 %)	Val (30 %)
Ripe (maduro)	400	280	120
Unripe (inmaduro)	400	280	120
Damaged (dañado)	400	280	120
Total	1.200	840	360

III-B. Extracción de características

La extracción de características se diseñó para capturar las propiedades visuales más discriminativas entre los tres estados del tomate. El preprocesamiento inicial consiste en detectar el objeto de interés mediante umbralización sobre píxeles

con intensidad mayor a 10 en cualquier canal, recortar la región delimitadora (*bounding box*) y redimensionar la imagen resultante a 224×224 píxeles. A continuación se computan los siguientes grupos de características:

III-B1. Estadísticas de color HSV y LAB: Para cada canal de los espacios de color HSV y CIELab se calculan cuatro estadísticas descriptivas sobre los píxeles del objeto: media, desviación estándar, percentil 25 y percentil 75. Esto produce $3 \times 4 + 3 \times 4 = 24$ características de estadísticas de color. El espacio HSV es particularmente discriminativo para madurez (dominancia de matiz rojo vs verde), mientras que LAB proporciona una representación perceptualmente uniforme que captura luminosidad y crominancia de forma independiente.

III-B2. Histogramas de cromaticidad HSV: Se computan histogramas normalizados de los tres canales HSV utilizando 16 bins por canal (16 bins para H en rango [0,180] y 16 bins para S y V en [0,255]), lo que aporta $16 \times 3 = 48$ características adicionales.

III-B3. Texturas GLCM: La matriz de coocurrencia de niveles de gris se calcula con distancias [1, 2] y ángulos [0°, 45°], y se extraen cinco propiedades: contraste, correlación, energía, homogeneidad y disimilaridad. Para cada propiedad se calculan la media y la desviación estándar sobre las cuatro combinaciones distancia-ángulo, resultando en $5 \times 2 = 10$ características de textura.

III-B4. Patrones binarios locales (LBP): Se aplica LBP uniforme con radio 1 y 8 puntos de muestreo sobre los píxeles del objeto, generando un histograma de 10 bins que captura patrones de microestructura superficial.

III-B5. Descriptores de defectos: Se incluyen dos características de baja complejidad pero alta relevancia para la clase *damaged*: (a) razón de puntos oscuros (*dark spot ratio*): fracción de píxeles del objeto con valor $V < 50$ en HSV; (b) razón de manchas marrones (*brown spot ratio*): fracción de píxeles en el rango HSV correspondiente a tonalidades marrones [H: 5–25, S: 40–255, V: 20–200].

III-B6. Densidad de bordes: Se aplica el detector de bordes Canny (umbrales 100 y 200) sobre la imagen en escala de grises y se calcula la proporción de píxeles de borde respecto al total del objeto.

El vector de características final de este enfoque (denominado *Enfoque 1: Imagen completa*) contiene **100 dimensiones**.

III-C. Estrategia de análisis por cuadrantes (Enfoque 2)

Para incorporar información espacial local, se implementó una estrategia de división de la imagen en cuatro bloques de igual tamaño (2×2). Sobre cada cuadrante se extrae el mismo conjunto de características descrito previamente. Posteriormente, los vectores obtenidos de los cuatro bloques son promediados elemento a elemento para construir un descriptor espacial agregado de 100 dimensiones. Finalmente, este descriptor local promedio se concatena con el vector global de la imagen completa, produciendo un vector final de **200 características**. Esta estrategia permite incorporar información espacial local sin incrementar excesivamente la dimensionalidad del problema.

III-D. Modelos evaluados

Todos los modelos fueron entrenados sobre los mismos conjuntos de entrenamiento (840 muestras) y validados sobre el mismo conjunto de validación (360 muestras). Las características fueron normalizadas mediante estandarización (media cero, varianza unitaria) antes de alimentar los modelos que son sensibles a la escala (SVM y KNN).

III-D1. Support Vector Machine (SVM-RBF): SVM con kernel de función de base radial (RBF), parámetros $C=10$ y $\gamma=scale$. El kernel RBF permite la clasificación no lineal en espacios de alta dimensión, siendo adecuado para vectores de características densas como los utilizados.

III-D2. Random Forest (RF): Ensamble de 200 árboles de decisión con criterio de pureza Gini. La naturaleza ensamblada del modelo le confiere robustez frente al ruido en las características y reduce el sobreajuste respecto a un árbol individual.

III-D3. Gradient Boosting (GB): Modelo de boosting con 200 estimadores que aprende de manera secuencial corrigiendo los errores del estimador anterior. Es el modelo más expresivo del conjunto pero también el más costoso computacionalmente.

III-D4. K-Nearest Neighbors (KNN): Clasificador basado en los 5 vecinos más cercanos bajo distancia euclidiana. Su simplicidad conceptual lo hace útil como línea base, aunque es sensible a la maldición de la dimensionalidad.

III-D5. MobileNetV3-Small (Deep Learning): Red neuronal convolucional preentrenada sobre ImageNet, adaptada mediante transferencia de aprendizaje de dos fases: (1) *fase de calentamiento* — 5 épocas entrenando únicamente la cabeza clasificadora con backbone congelado ($lr=1e-3$); (2) *fine-tuning* — 10 épocas descongelando los últimos bloques del backbone ($lr=5e-5$ para capas superiores, $1e-4$ para la cabeza). Se empleó aumentación de datos durante el entrenamiento: volteo horizontal/vertical, rotación aleatoria de $\pm 15^\circ$ y jitter de color (brillo ± 0.2 , contraste ± 0.2 , saturación ± 0.3).

IV. DISEÑO DEL SISTEMA

IV-A. Arquitectura general

RipenTom se concibe como un sistema de dos componentes desacoplados: un **backend de inferencia** expuesto como API REST y un **frontend web estático** que consume dicha API. Esta separación permite escalar o actualizar cada componente de forma independiente, y facilita la integración futura con otros clientes (aplicaciones móviles, sistemas embebidos).

IV-B. Backend: Flask-API

El backend está implementado en Python utilizando Flask como microframework. Expone un endpoint principal POST /predict que acepta una imagen como archivo multipart, ejecuta la pipeline de preprocesamiento y extracción de características, invoca el modelo Random Forest serializado (200 estimadores, 100 features), y retorna la clase predicha junto con las probabilidades por clase en formato JSON. El modelo y el escalador se cargan desde archivos .joblib al inicio del servidor. La pipeline implementada en producción corresponde al Enfoque 1 (vector de 100 características sobre imagen

completa), dado que el Random Forest con este vector ofrece el mejor balance entre rendimiento y latencia de inferencia.

```
[Usuario sube imagen]
|
v
[Frontend HTML/JS - Vercel]
| POST /predict (multipart)
v
[Flask API Backend]
- Preprocesamiento (crop, resize 224x224)
- Extraccion de features (100 dims)
- Normalizacion (StandardScaler)
- Inferencia (Random Forest, 200 trees)
|
v
[JSON: {clase, probabilidades}]
|
v
[Frontend muestra resultado]
```

Figura 1. Flujo de inferencia de RipenTom

IV-C. Frontend: HTML estático en Vercel

El frontend consiste en una aplicación web de página única desarrollada con HTML, CSS y JavaScript vanilla, sin dependencias de framework. Permite al usuario cargar una imagen desde su dispositivo mediante un formulario de selección de archivo, previsualizar la imagen antes de enviarla, y obtener la clasificación del sistema junto con el porcentaje de confianza para cada clase. La interfaz está desplegada en Vercel, aprovechando su red de distribución de contenido (CDN) global para minimizar la latencia de carga.

IV-D. Serialización y despliegue del modelo

El modelo Random Forest entrenado fue serializado mediante la biblioteca `joblib` junto con el objeto `StandardScaler` utilizado durante el entrenamiento. El modelo final contiene 200 estimadores, opera sobre vectores de 100 dimensiones y maneja tres clases codificadas numéricamente (0: damaged, 1: ripe, 2: unripe). La verificación post-serIALIZACIÓN confirma que la dimensión del vector de entrada y las probabilidades de salida son consistentes con el entrenamiento.

V. RESULTADOS

V-A. Desempeño del modelo de aprendizaje profundo (MobileNetV3)

El proceso de entrenamiento de MobileNetV3-Small mostró convergencia progresiva. Durante la Fase 1 (backbone congelado, 5 épocas), la accuracy de validación mejoró de 0.358 en la primera época hasta 0.758 al finalizar, evidenciando la capacidad de las representaciones preentrenadas de ImageNet como punto de partida. La Fase 2 (fine-tuning, 10 épocas) logró mejorar este valor hasta un máximo de **0.783** de accuracy de validación, obtenido en la época 9.

Cuadro II
REPORTE DE CLASIFICACIÓN / MOBILENETV3-SMALL

Clase	Precision	Recall	F1	Support
Damaged	0.83	0.66	0.73	120
Unripe	0.86	0.93	0.89	120
Ripe	0.68	0.77	0.72	120
Macro avg	0.79	0.78	0.78	360

La clase *unripe* fue la mejor clasificada, probablemente porque su coloración verde uniforme contrasta claramente con las otras dos clases. Las clases *damaged* y *ripe* presentaron mayor confusión entre sí, dado que ambas pueden exhibir tonalidades rojizas similares.

V-B. Desempeño de modelos ML clásicos — Enfoque 1 (100 features)

Con el vector de 100 características extraído de la imagen completa, todos los modelos de ML clásico superaron al modelo de aprendizaje profundo. El mejor modelo en este enfoque fue **Random Forest** con accuracy de 0.853 y F1-macro de 0.853.

Cuadro III
COMPARATIVA DE MODELOS — ENFOQUE 1 (IMAGEN COMPLETA, 100 FEATURES)

Modelo	Accuracy	F1-macro	Tiempo
Random Forest	0.853	0.853	1.1s
Gradient Boosting	0.850	0.850	5.9s
SVM (RBF)	0.847	0.845	0.1s
KNN (k=5)	0.819	0.817	0.0s
MobileNetV3 (DL)	0.794	0.795	—

Cuadro IV
REPORTE POR CLASE / RANDOM FOREST, ENFOQUE 1

Clase	Precision	Recall	F1	Support
Damaged	0.89	0.85	0.87	120
Unripe	0.89	0.90	0.89	120
Ripe	0.79	0.81	0.80	120
Macro avg	0.85	0.85	0.85	360

V-C. Desempeño de modelos ML clásicos / Enfoque 2 (cuadrantes 2x2, 200 features)

La incorporación del análisis por cuadrantes incrementó el vector de características a 200 dimensiones, aportando información espacial local. En este escenario, **SVM-RBF** se convirtió en el modelo con mejor desempeño global, alcanzando una accuracy de **0.867** y F1-macro de **0.865**.

El impacto de los cuadrantes es diferencial por clase: la clase *unripe* experimentó el mayor beneficio (F1: 0.89 → 0.95), mientras que *damaged* mejoró moderadamente (F1: 0.87 → 0.84 en RF, pero con SVM: 0.84). La clase *ripe* se mantuvo relativamente estable. El incremento marginal en el rendimiento general (1.5 pp en accuracy) sugiere que la

Cuadro V
COMPARATIVA DE MODELOS — ENFOQUE 2 (CUADRANTES 2x2, 200
FEATURES)

Modelo	Accuracy	F1-macro	Tiempo
SVM (RBF)	0.867	0.865	0.1s
Random Forest	0.856	0.856	1.7s
Gradient Boosting	0.847	0.847	12.6s
KNN (k=5)	0.822	0.820	0.1s
MobileNetV3 (DL)	0.783	0.782	—

Cuadro VI
REPORTE POR CLASE / SVM-RBF, ENFOQUE 2

Clase	Precision	Recall	F1	Support
Damaged	0.86	0.82	0.84	120
Unripe	0.91	0.99	0.95	120
Ripe	0.82	0.78	0.80	120
Macro avg	0.86	0.87	0.86	360

información espacial local aporta valor principalmente para la discriminación de *unripe*, cuya uniformidad de color por cuadrante es muy distintiva.

VI-D. Comparativa global y selección del modelo para despliegue

Cuadro VII
RESUMEN COMPARATIVO DE TODOS LOS ENFOQUES

Enfoque	Modelo	Acc.	F1
DL (MobileNetV3)	Fine-tuning	0.783	0.782
E1 — 100 features	KNN (k=5)	0.819	0.817
E1 — 100 features	SVM (RBF)	0.847	0.845
E1 — 100 features	Gradient Boost.	0.850	0.850
E1 — 100 features	Random Forest	0.853	0.853
E2 — 200 features	Gradient Boost.	0.847	0.847
E2 — 200 features	Random Forest	0.856	0.856
E2 — 200 feat.	SVM (RBF)	0.867	0.865

Para el despliegue en producción se seleccionó el modelo **Random Forest con Enfoque 1 (100 features)**, en lugar del SVM con cuadrantes que es técnicamente superior. Esta decisión se justifica por razones de ingeniería: (a) el tiempo de extracción de características se duplica con el Enfoque 2, lo que impacta la latencia de la API; (b) la diferencia de rendimiento entre ambos modelos es pequeña (cerca de 1.5 pp); y (c) Random Forest provee estimaciones de probabilidad por clase directamente (`predict_proba`), lo que enriquece la respuesta de la API para el usuario. El modelo serializado consta de 200 estimadores y opera sobre vectores de 100 dimensiones.

VI. DISCUSIÓN ÉTICA

VI-A. Declaración de uso de inteligencia artificial

En el desarrollo de este proyecto se utilizaron herramientas de inteligencia artificial generativa como apoyo en tareas específicas: (a) depuración de código Python y sugerencias

de implementación para la pipeline de extracción de características; (b) consulta de documentación técnica de bibliotecas (scikit-learn, PyTorch, OpenCV); (c) revisión gramatical y ortográfica del texto en español. Todo el diseño conceptual del sistema, la toma de decisiones sobre arquitectura, la curación del dataset, la interpretación de resultados y las conclusiones son producto del trabajo original del equipo. Las herramientas de IA fueron utilizadas como apoyo instrumental, no como sustituto del juicio crítico y la comprensión técnica del equipo.

VI-B. Autonomía del sistema y alcance de sus decisiones

RipenTom opera como un sistema de apoyo a la decisión, no como un agente autónomo con capacidad de acción directa sobre procesos físicos. La autonomía del sistema se limita a emitir una clasificación (ripe/unripe/damaged) y probabilidades asociadas a partir de una imagen proporcionada por el usuario. Cualquier decisión de acción subsiguiente (descartar un lote, enrutar producto para procesamiento, informar a un operario) recae en un ser humano que interpreta el resultado. Esta separación explícita entre inferencia y actuación es una decisión de diseño deliberada para mantener la supervisión humana en el lazo de control.

VI-C. Riesgos y consecuencias de fallo

Un error del sistema en contexto agroindustrial puede tener dos consecuencias asimétricas: (a) *falso negativo sobre daño*: clasificar como maduro un tomate dañado implica que producto en mal estado alcance al consumidor, con riesgos sanitarios potenciales; (b) *falso positivo sobre daño*: clasificar como dañado un tomate maduro implica descarte innecesario, con impacto económico para el productor. El análisis de los reportes de clasificación muestra que la clase *damaged* presenta el recall más bajo en la mayoría de los modelos (0.66 en MobileNetV3, 0.82–0.85 en ML clásico), lo que significa que el sistema tiende a *no detectar* algunos casos de daño. En contextos de alta exigencia sanitaria, este riesgo debe ser explícitamente comunicado a los operadores del sistema y complementado con inspección humana para las muestras de baja confianza.

El sistema incluye estimaciones de probabilidad por clase que permiten implementar umbrales de confianza: rechazos automáticos solo cuando la confianza supere un umbral configurable, derivando a revisión humana los casos inciertos.

VI-D. Sesgos potenciales del dataset

El dataset curado, aunque mejorado respecto al original, puede contener sesgos no identificados: (a) *sesgo de dominio*: las imágenes proceden de fuentes controladas con fondo negro y condiciones de iluminación relativamente uniformes; en entornos reales (luz natural variable, fondos complejos), el rendimiento del sistema puede degradarse significativamente; (b) *sesgo de variedad*: el modelo fue entrenado sobre imágenes de variedades específicas de tomate; podría generalizar mal a variedades de formas o coloraciones inusuales; (c) *sesgo de representación del daño*: la categoría *damaged* engloba tipos heterogéneos de defectos (golpes, pudrición, enfermedades),

cuya representación en el dataset puede no ser proporcional a su prevalencia real en campo.

Estos sesgos son inherentes a cualquier sistema de aprendizaje automático entrenado con datos históricos y deben ser considerados al momento de evaluar la idoneidad del sistema para un contexto de despliegue específico.

VI-E. Impacto sobre los usuarios

El despliegue de RipenTom como herramienta web pública implica que usuarios sin formación técnica pueden someter imágenes a clasificación y recibir un resultado que perciban como definitivo. Para mitigar el riesgo de sobreconfianza en el sistema, la interfaz muestra explícitamente las probabilidades por clase, comunica la naturaleza estadística de la predicción y recomienda confirmación humana en casos de baja confianza. El sistema no almacena las imágenes enviadas por los usuarios, respetando la privacidad de los datos.

Desde una perspectiva más amplia, la automatización de la clasificación de calidad puede impactar los roles laborales en la industria agrícola. Si bien el sistema no está diseñado para reemplazar puestos de trabajo sino para apoyar y agilizar la inspección manual, es responsabilidad de quienes lo adopten gestionar este impacto de forma justa para los trabajadores afectados.

VII. CONCLUSIONES

Este trabajo presentó RipenTom, un sistema de clasificación automatizada del estado de madurez de tomates que demuestra que la ingeniería cuidadosa de características puede superar a arquitecturas de aprendizaje profundo en dominios de datos limitados y controlados.

Las principales conclusiones del trabajo son:

- **La calidad del dataset es el factor dominante:** la curación manual del conjunto de datos, con corrección de etiquetas erróneas y eliminación de duplicados, fue la intervención de mayor impacto en el rendimiento del sistema, más que cualquier decisión de arquitectura.
- **ML clásico supera a DL con datos limitados:** todos los modelos de aprendizaje automático clásico superaron a MobileNetV3 en accuracy (0.819–0.867 vs 0.783), confirmando que las redes profundas requieren grandes volúmenes de datos para materializar su capacidad representacional.
- **El análisis por cuadrantes aporta valor:** la estrategia de división 2x2 incrementó el desempeño de todos los modelos, con mejoras más pronunciadas para la clase *unripe*, al capturar la distribución espacial de las características cromáticas y de textura.
- **Random Forest como modelo de producción:** la combinación de buen rendimiento, velocidad de inferencia y capacidad de proveer estimaciones de probabilidad hace a Random Forest la opción más equilibrada para el despliegue web en tiempo real.

Como trabajo futuro se propone: (a) ampliar el dataset con imágenes capturadas en condiciones reales de campo; (b) explorar técnicas de aprendizaje semi-supervisado para

reducir la dependencia de etiquetado manual; (c) evaluar la incorporación de detectores de objetos para segmentar automáticamente el tomate del fondo en imágenes no controladas; (d) implementar monitoreo de distribución en producción (*data drift detection*) para detectar degradación del rendimiento.

REFERENCIAS

- [1] Brosnan, Tadhg y Sun, Da-Wen. (2004). Improving quality inspection of food products by computer vision—A review. *Journal of Food Engineering*. 61. 3-16. 10.1016/S0260-8774(03)00183-3.
- [2] A. Alarcón Zayas, “Calidad poscosecha del tomate (*Solanum lycopersicum* L.) cultivado en sistemas ecológicos de fertilización,” Tesis doctoral, Escuela Técnica Superior de Ingenieros Agrónomos, Universidad Politécnica de Madrid, Madrid, España, 2013. [En línea]. Disponible en: https://oa.upm.es/21908/1/ALEJANDRO_ALARCON_ZAYAS.pdf
- [3] Bhargava, Anuja y Bansal, Atul. (2018). Fruits and Vegetables Quality Evaluation Using Computer Vision: A Review. *Journal of King Saud University - Computer and Information Sciences*. 33. 10.1016/j.jksuci.2018.06.002.
- [4] Kondo, Naoshi y Ahmad, Usman y Monta, Mitsui y Murase, Haruhiko. (2000). Machine vision based quality evaluation of Iokan orange fruit using neural networks. *Computers and Electronics in Agriculture*. 29. 135-147. 10.1016/S0168-1699(00)00141-1.
- [5] Payne, Alison y Walsh, Kerry y Subedi, Phul y Jarvis, Dennis. (2014). Estimating mango crop yield using image analysis using fruit at ‘stone hardening’ stage and night time imaging. *Computers and Electronics in Agriculture*. 100. 160–167. 10.1016/j.compag.2013.11.011.
- [6] R. M. Haralick, K. Shanmugam, y I. Dinstein, “Textural features for image classification,” *IEEE Transactions on Systems, Man, and Cybernetics*, vol. SMC-3, no. 6, pp. 610–621, 1973, doi: 10.1109/TSMC.1973.4309314.
- [7] T. Ojala, M. Pietikäinen, y T. Mäenpää, “Multiresolution gray-scale and rotation invariant texture classification with local binary patterns,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 24, no. 7, pp. 971–987, 2002, doi: 10.1109/TPAMI.2002.1017623.
- [8] J. Blasco, N. Aleixos, y E. Moltó, “Machine vision system for automatic quality grading of fruit,” *Biosystems Engineering*, vol. 85, no. 4, pp. 415–423, 2003, doi: 10.1016/S1537-5110(03)00088-6.
- [9] Naranjo-Torres, J., Mora, M., Hernández-García, R., Barrientos, R. J., Fredes, C., y Valenzuela, A. (2020). A Review of Convolutional Neural Network Applied to Fruit Image Processing. *Applied Sciences*, 10(10), 3443. <https://doi.org/10.3390/app10103443>
- [10] Mohanty SP, Hughes DP and Salathé M (2016) Using Deep Learning for Image-Based Plant Disease Detection. *Front. Plant Sci*. 7:1419. doi: 10.3389/fpls.2016.01419
- [11] Kamilaris, Andreas, Prenafeta Boldú, Francesc. (2018). Deep Learning in Agriculture: A Survey. *Computers and Electronics in Agriculture*. 147. 10.1016/j.compag.2018.02.016.
- [12] A. Howard, M. Sandler, G. Chu, L.-C. Chen, B. Chen, M. Tan, W. Wang, Y. Zhu, R. Pang, V. Vasudevan, Q. V. Le, y H. Adam, “Searching for MobileNetV3,” en *Proc. IEEE/CVF International Conference on Computer Vision (ICCV)*, 2019, pp. 1314–1324.
- [13] Ding, C., Pereira, T., Xiao, R., Lee, R. J., y Hu, X. (2022). Impact of Label Noise on the Learning Based Models for a Binary Classification of Physiological Signal. *Sensors*, 22(19), 7166. <https://doi.org/10.3390/s22197166>
- [14] He, Haibo y Garcia, E.A.. (2009). Learning from Imbalanced Data. *Knowledge and Data Engineering, IEEE Transactions on*. 21. 1263 - 1284. 10.1109/TKDE.2008.239.
- [15] L. Perez y J. Wang, “The effectiveness of data augmentation in image classification using deep learning,” *arXiv preprint arXiv:1712.04621*, 2017.
- [16] M. T. Riquelme, P. Barreiro, M. Ruiz-Altisent, y C. Valero, “Olive classification according to external damage using image analysis,” *Journal of Food Engineering*, vol. 87, no. 3, pp. 371–379, 2008.
- [17] J. Singh, “Tomato Classification: Unripe, Ripe and Damaged,” *Kaggle Dataset*, 2023. [Online]. Available: <https://www.kaggle.com/datasets/jasroop11/tomato-classification-unripe-ripe-and-damaged>